

Healthcare Appointment & Follow-up Manager

System Design & Technical Documentation

A role-based full-stack platform connecting patients, doctors and administrators

React + Vite · FastAPI · PostgreSQL · SQLAlchemy · Groq LLM · Google Calendar API

1. Project Overview

The Healthcare Appointment & Follow-up Manager is a full-stack healthcare appointment management platform designed to connect patients, doctors, and administrators through separate, role-based workflows.

Beyond basic appointment booking, the platform supports doctor availability and leave management, appointment lifecycle management, pre-visit symptom capture, notification processing, and Google Calendar synchronization.

Patients can find doctors, check available appointment slots, submit information before a consultation, and manage their own appointments. Doctors can manage their appointment workflow and review relevant patient information ahead of each consultation. Administrators manage doctor profiles, working hours, slot duration, and doctor leave.

2. Live Deployment

The application is deployed on Render, with the frontend and backend hosted as separate services.

[Frontend application](#)

[Backend API](#)

[Swagger / API documentation](#)

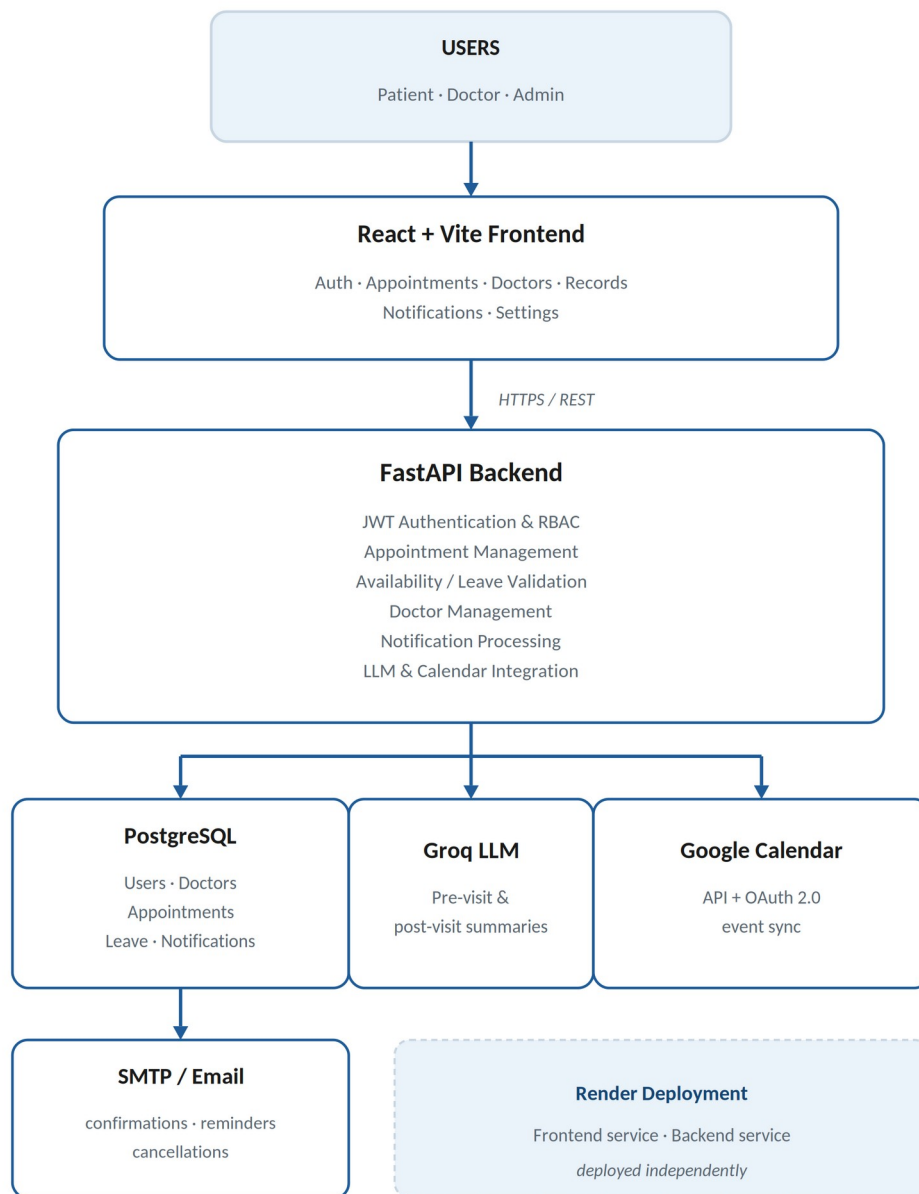
[Alternate backend deployment](#)

[GitHub repository](#)

3. System Architecture

The frontend is implemented with React and Vite. The backend is a Python FastAPI service exposing REST APIs. PostgreSQL provides persistent storage, with SQLAlchemy as the ORM layer and Alembic managing schema migrations. Authentication and authorization are handled through JWT-based role access for patients, doctors and administrators.

Three external services extend the platform: the Groq LLM for pre-visit and post-visit summaries, SMTP for email notifications, and the Google Calendar API (via OAuth 2.0) for calendar synchronization.



4. Technology Stack

Layer	Technology
Frontend	React
Frontend Build Tool	Vite
Backend	Python, FastAPI
API Style	REST API
Database	PostgreSQL
ORM	SQLAlchemy
Database Migration	Alembic

Layer	Technology
Authentication	JWT
Authorization	Role-based access control
LLM	Groq API
Email	SMTP
Calendar	Google Calendar API
Calendar Authentication	OAuth 2.0
API Documentation	Swagger UI / OpenAPI
Deployment	Render
Version Control	Git / GitHub

5. Roles and Features

5.1 Patient

- Registration and login
- Search doctors, including by specialization
- View doctor availability
- Book, cancel and reschedule appointments
- Provide pre-visit symptom information
- View appointment details
- Receive appointment notifications
- Calendar synchronization

5.2 Doctor

- Doctor-specific appointment workflow
- Availability based on configured working hours
- Slot-based appointment management
- Leave management
- View scheduled appointments
- Access information submitted before a consultation
- Post-visit workflow

5.3 Admin

- Manage doctor profiles
- Configure specialization, working hours and slot duration
- Manage doctor leave and availability

6. Appointment Booking Workflow

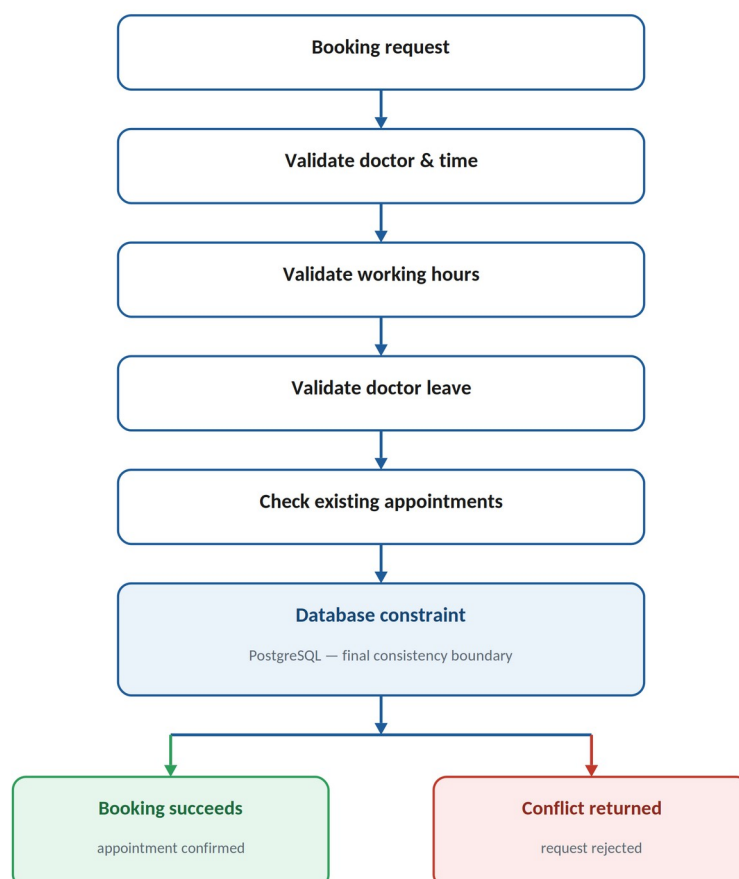
Before an appointment is created, the backend validates, in order:

- The doctor exists
- The requested date and time are valid
- The requested time falls within working hours
- The doctor is not on leave
- The slot is available
- Existing appointments do not conflict

Only after these checks pass does the appointment become confirmed, which reduces invalid bookings and race-condition problems.

7. Double-Booking Prevention

A simple frontend availability check is not sufficient on its own, because two patients can see the same slot as available and submit booking requests at nearly the same time. The backend therefore validates the doctor, working hours, leave status and existing appointments before creating a booking — and PostgreSQL constraints provide the final consistency boundary.



Even if two concurrent requests both pass the initial availability check, the database prevents conflicting records from being created: one transaction succeeds and the other is rejected. A key

implementation insight was that checking availability in application code alone cannot fully solve this race condition — the database has to enforce the final invariant.

8. Slot Hold Behaviour

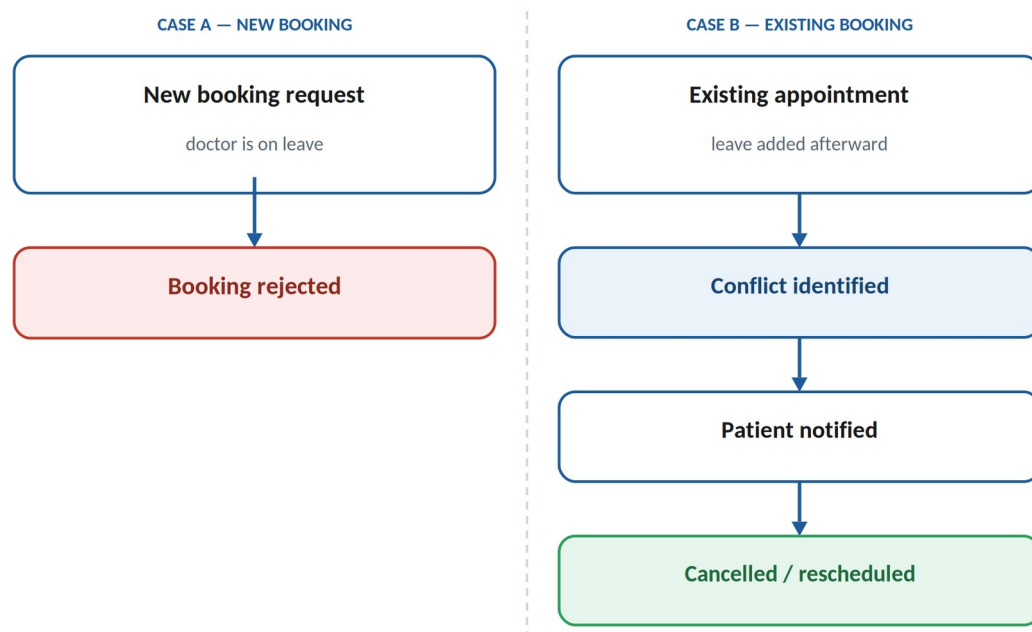
The symptom form introduces a related challenge: a patient may select a slot and then spend time entering symptoms before confirming the booking. The system distinguishes between a slot that is displayed as available and one that is genuinely reserved.

The current implementation relies on backend and database consistency rather than treating a frontend selection as a permanent reservation. This prevents abandoned browser sessions from permanently blocking a slot, since a slot should not become unavailable simply because a user opened the booking form.

A future enhancement under consideration is a dedicated, temporary `slot_holds` record with an expiry timestamp, with expired holds released automatically.

9. Doctor Leave and Booking Conflicts

Doctor working hours and leave days are factored into availability calculations. If a doctor is marked unavailable due to leave, the affected slots cannot be booked. The system also has to handle leave that is added after appointments already exist.



When leave is added retroactively, the system identifies the appointments it affects so that patients can be notified and the appointments cancelled or rescheduled. Leave validation happens entirely on the backend, so it cannot be bypassed by modifying frontend requests. This keeps the availability system consistent with the doctor's actual schedule.

10. Rescheduling and Cancellation

Appointments move through a lifecycle of states: Booked, Confirmed, Cancelled, and Completed.

10.1 Rescheduling

- The new slot is validated
- Availability is checked
- The existing appointment is updated
- Notifications are processed
- The corresponding Google Calendar event is updated

10.2 Cancellation

- Appointment status is updated
- Notification processing is triggered
- The associated calendar event is deleted where applicable

11. LLM Integration (Groq)

The Groq LLM converts raw clinical information into concise, structured summaries for two workflows: a pre-visit symptom summary and a patient-friendly post-visit summary. In both cases, the original clinical input — patient symptoms or doctor notes — remains the source of truth, and generated summaries are stored separately as derived data.

11.1 Pre-visit summary

The prompt asks the model to analyse submitted symptoms and return an urgency level (Low, Medium or High), the chief complaint, and three suggested questions for the doctor:

```
Analyse these symptoms and return:  
urgency level (Low / Medium / High),  
chief complaint,  
and three suggested questions for the doctor.
```

```
Symptoms:  
<symptoms>
```

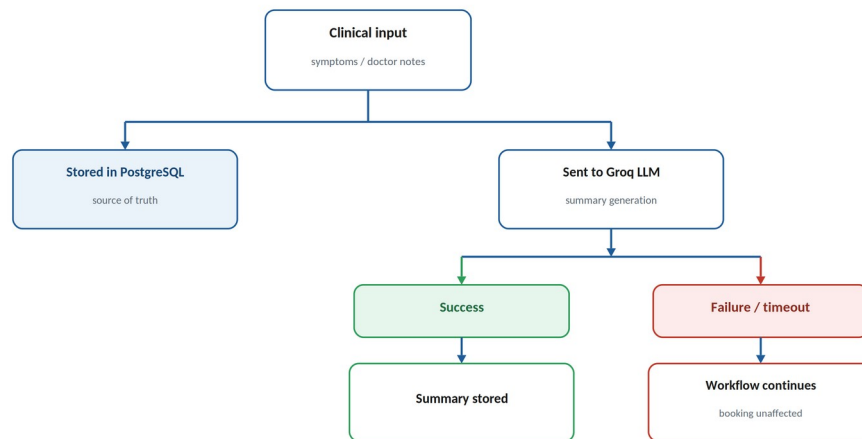
11.2 Post-visit summary

After a consultation, doctor notes and prescription details are converted into a patient-friendly summary that includes a medication schedule and follow-up steps:

```
Convert these clinical notes into a patient-friendly summary  
with medication schedule and follow-up steps:
```

```
<notes>
```

11.3 Failure handling



Because generated summaries are stored separately from the source clinical data, an LLM failure, timeout, or invalid response does not interrupt the appointment workflow. The LLM is treated as an assisting service that enhances the platform rather than one it depends on for core functionality.

12. Notifications

The backend triggers notifications for key appointment events: booking confirmation, cancellation, rescheduling, appointment reminders, and doctor or patient updates. Email delivery uses SMTP.

Notification processing is kept separate from the core appointment transaction. A successful database booking is not rolled back simply because an email provider is temporarily unavailable — failed notifications can be retried independently, which keeps PostgreSQL as the system's source of truth.

13. Google Calendar Integration

Calendar synchronization uses the Google Calendar API with OAuth 2.0, so that calendar operations are performed under the appropriate user's authorization.

- Booking: a corresponding calendar event is created
- Rescheduling: the existing calendar event is updated
- Cancellation: the corresponding calendar event is deleted

Calendar synchronization is treated as a downstream operation. If the Google Calendar API is temporarily unavailable, the appointment remains stored correctly in PostgreSQL, so a calendar outage does not affect the core booking system.

14. Database and API Design

PostgreSQL stores users, roles, doctors, working hours, leave information, appointments, appointment status, patient-provided information, generated summaries, notification information, and calendar-related information. A unique email constraint is enforced for users.

SQLAlchemy provides the ORM layer, and Alembic manages schema migrations, typically applied with:


```
alembic upgrade head
```

FastAPI provides REST endpoints with request validation, authentication and role-based authorization, and automatically generates OpenAPI and Swagger UI documentation, making the API straightforward to test and integrate with the React frontend.

15. Authentication and Authorization

The application follows a role-based architecture with three roles: patient, doctor and admin. JWT-based authentication protects API operations, and role checks ensure each user only accesses operations appropriate to their role.

Role	Scope of access
Patient	Appointments and patient information
Doctor	Doctor appointments and consultation workflow
Admin	Doctor, profile and availability management

16. Error Handling and Reliability Principle

The appointment system separates core database operations from external integrations, so that a failure in one of the following does not unnecessarily roll back a valid appointment:

- LLM processing (Groq)
- Email notifications (SMTP)
- Google Calendar synchronization

The overarching design principle is that PostgreSQL is the source of truth. Appointment creation, cancellation, rescheduling and availability decisions are controlled entirely by the backend and database, while Groq, SMTP and Google Calendar enhance the platform without controlling its core state. This provides safe appointment booking, doctor leave conflict handling, reliable notification processing, and graceful degradation when external integrations fail.

17. Project Structure

```
healthcare-appointment-manager/
|
+-- backend/
|   +-- app/
|       |-- main.py
|       |-- models/
|       |-- schemas/
|       |-- routers/
|       |-- services/
|       |-- database/
|   +-- requirements.txt
|   +-- alembic.ini
|   +-- alembic/
|
+-- frontend/
|   +-- src/
|   +-- public/
```

```
|   +-- package.json
|   +-- vite.config.*
|
+-- prisma/
|
+-- .env.example
+-- README.md
```

The main FastAPI application entry point is `backend/app/main.py`.

18. Environment Variables

Configuration is provided through a `.env` file based on `.env.example`. Typical variables include:

```
DATABASE_URL=
JWT_SECRET_KEY=

GROQ_API_KEY=

SMTP_HOST=
SMTP_PORT=
SMTP_USERNAME=
SMTP_PASSWORD=
SMTP_FROM_EMAIL=

GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=
```

Secrets should never be committed to GitHub; required variables are configured directly in the Render service for deployed environments.

19. Local Setup

19.1 Backend

```
git clone https://github.com/nehariona/healthcare-appointment-manager.git
cd healthcare-appointment-manager

python -m venv venv
source venv/bin/activate

pip install -r backend/requirements.txt

cp .env.example .env

cd backend
alembic upgrade head

uvicorn app.main:app --reload
```

The local API is available at `http://127.0.0.1:8000`, with Swagger UI at `http://127.0.0.1:8000/docs`.

19.2 Frontend

```
cd frontend
npm install
npm run dev
```

The frontend runs at <http://localhost:5173> and communicates with the FastAPI backend through REST API calls.

20. Render Deployment

The backend is deployed on Render with the root directory set to backend and the following start command:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

The frontend is deployed as a separate Render service. Required environment variables are configured in each Render service rather than committed to the repository.

21. Google Calendar Setup

- Open Google Cloud Console
- Create or select a Google Cloud project
- Enable the Google Calendar API
- Configure the OAuth consent screen
- Create OAuth 2.0 credentials
- Configure the authorized redirect URI
- Add the generated client ID and client secret to the environment variables
- Complete the OAuth authorization flow
- Store the resulting calendar information required by the application

22. Key Engineering Challenges

The main challenges encountered during development were:

- Preventing double-booking during concurrent requests
- Correctly validating working hours and doctor leave
- Handling existing appointments when leave is introduced later
- Designing LLM prompts that return structured, useful summaries
- Ensuring LLM failures do not break the appointment workflow
- Integrating Google OAuth and the calendar event lifecycle
- Keeping email and calendar failures separate from database transactions
- Maintaining consistency between the React frontend and FastAPI backend
- Deploying frontend and backend separately on Render with correct environment configuration

These challenges shaped the final architecture: the database remains the source of truth, while external services are treated as independently recoverable integrations.

23. Project Status

Implemented areas include:

- FastAPI backend with PostgreSQL, SQLAlchemy and Alembic
- Appointment availability logic, working-hour validation and doctor leave handling
- Booking, cancellation and rescheduling
- PostgreSQL-based double-booking protection
- Google Calendar create / update / delete workflow
- Notification processing
- React + Vite frontend
- Render backend deployment
- Swagger / OpenAPI documentation

LLM, email, calendar, and background-processing capabilities depend on their corresponding environment configuration and the availability of each external service.